

Fișă de lucru – Șiruri de caractere

Realizată de: Aloman Denisa-Maria

Teorie

1.1 Ce este un șir de caractere?

Un **șir de caractere** (*string*) este o secvență ordonată de caractere delimitate de ghilimele simple sau duble. Șirurile sunt **imutabile**: nu pot fi modificate direct după creare.

Există mai multe moduri de a delimita un șir:

- **Ghilimele duble**: "Salut"
- **Apostrofuri**: 'Salut'
- **Ghilimele/Apostrofuri triple** (`"""..."""` sau `'''...'''`): folosite pentru șiruri pe mai multe linii

```
s1 = "Salut, ce mai faci?"      # ghilimele duble
s2 = 'Python'                  # apostrofuri
s3 = """Aceasta este
o propozitie pe
mai multe linii."""           # ghilimele triple
```

Observație: Dacă vrem să definim un șir care conține un apostrof, îl scriem între ghilimele duble, și invers.

```
s4 = "Dom'le" # apostrof in interior -> ghilimele duble
s5 = 'Eu spun "salut".' # ghilimele in interior -> apostrofuri
```

Putem afla **tipul** unei variabile cu funcția `type()`. În Python, tipul unui șir de caractere se numește `str` (prescurtare de la *string*).

```
s1 = "Salut, ce mai faci?"
print(type(s1)) # afiseaza: <class 'str'>
                # variabila s1 este de tipul str (sir de caractere)
```

1.2 Indexare și acces la caractere

Un șir de caractere poate fi privit ca o **listă de caractere** - fiecare caracter ocupă o poziție fixă, numită **index**, și poate fi accesat individual. Indexarea începe de la 0. Se pot folosi și **indecși negativi**.

P	y	t	h	o	n
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

```
s = "Python"
print(s[0])    # P
print(s[-2])   # o
```

1.3 Slicing (segmentare)

Uneori, avem nevoie nu de un singur caracter, ci de o **secvență** dintr-un șir (de exemplu, primele 3 litere sau ultimele 2 caractere). Această operație se numește slicing (segmentare).

Sintaxa: `s[start : stop : step]`. Se extrage subșirul de la **start** (inclusiv) până la **stop** (exclusiv).

- Dacă **step** este negativ, parcurgerea este inversă.
- Dacă **start** lipsește, felierea începe de la primul caracter.
- Dacă **stop** lipsește, felierea continuă până la ultimul caracter.
- Dacă **step** lipsește, se consideră implicit valoarea 1.

```
s = "Informatica"
print(s[:5])    # Infor
print(s[5:])    # matica
print(s[2:5:2]) # fr
print(s[::])    # Informatica
```

1.4 Funcții built-in utile

Funcția	Ce face	Exemplu
<code>len(s)</code>	Calculează lungimea șirului	<code>len("Informatica") → 11</code>
<code>ord(c)</code>	Determină codul Unicode al caracterului	<code>ord("A") → 65</code>
<code>chr(n)</code>	Transformă un cod Unicode în caracter	<code>chr(65) → "A"</code>
<code>str(x)</code>	Transformă o valoare în șir de caractere	<code>str(123) → "123"</code>
<code>int(s)</code>	Transformă un șir în număr întreg	<code>int("123") → 123</code>

1.5 Metode uzuale ale șirurilor

Metoda	Ce face	Exemplu
<code>upper()</code>	Transformă în majuscule	<code>"abc".upper()</code> → <code>"ABC"</code>
<code>lower()</code>	Transformă în minuscule	<code>"ABC".lower()</code> → <code>"abc"</code>
<code>strip()</code>	Elimină spațiile de la capete	<code>" bun ".strip()</code> → <code>"bun"</code>
<code>split(sep)</code>	Împarte după separator, returnează listă	<code>"a,b,c".split(",")</code> → <code>['a','b','c']</code>
<code>replace(a,b)</code>	Înlocuiește a cu b	<code>"abab".replace("a","x")</code> → <code>"xbxb"</code>
<code>find(sub)</code>	Returnează indexul primei apariții a sub (-1 dacă nu există)	<code>"Python".find("t")</code> → <code>2</code>
<code>count(sub)</code>	Numără aparițiile unui subșir	<code>"banana".count("a")</code> → <code>3</code>
<code>startswith(s)</code>	Verifică dacă șirul începe cu s	<code>"Python".startswith("Py")</code> → <code>True</code>
<code>endswith(s)</code>	Verifică dacă șirul se termină cu s	<code>"Python".endswith("on")</code> → <code>True</code>
<code>isdigit()</code>	Returnează <code>True</code> dacă toate caracterele sunt cifre	<code>"123".isdigit()</code> → <code>True</code>
<code>isalpha()</code>	Returnează <code>True</code> dacă toate caracterele sunt litere	<code>"abc".isalpha()</code> → <code>True</code>

1.6 Parcurgerea unui șir

Putem parcurge fiecare caracter dintr-un șir folosind o instrucțiune repetitivă.

```
s = "Python"

for caracter in s:
    print(caracter)
```

Afișare

P
y
t
h
o
n

Putem parcurge și cu index, folosind funcțiile `range()` și `len()`:

```
s = "Python"

for i in range(len(s)):
    print(i, s[i])
```

Afișare

```
0 P
1 y
2 t
3 h
4 o
5 n
```

1.7 Concatenare și repetare

Concatenarea înseamnă lipirea a două șiruri cu operatorul `+`. **Repetarea** înseamnă multiplicarea unui șir cu operatorul `*`.

```
# Concatenare
s1 = "Hello"
s2 = " World"
s3 = s1 + s2
print(s3)  # Hello World

# Repetare
s4 = "ha"
print(s4 * 3)  # hahaha

# Concatenare in bucla
rezultat = ""
for i in range(1, 4):
    rezultat = rezultat + str(i)
print(rezultat)  # 123
```

Exemple rezolvate

Exemplul 1 - Afișează doar literele mari dintr-un șir

Cerință: Citește un șir și afișează doar caracterele care sunt litere mari.

Exemplu de rulare:

```
Input: "InfoRMatica"
Litere mari gasite: I, R, M
Output:
I
R
M
```

Caz particular:

```
Input: "informatica"
Output:
Programul nu afiseaza nimic, deoarece sirul dat nu contine
nicio litera mare
```

Rezolvare pas cu pas:

1. Citim șirul de la tastatură.
2. Parcurgem șirul caracter cu caracter.
3. Pentru fiecare caracter, verificăm dacă este literă mare. Dacă este literă mare, o afișăm.

```
s = input("Introduceti un sir: ") # pas 1: citim sirul
for caracter in s:                 # pas 2: parcurgem sirul
    if caracter.isupper():         # pas 3: verificam daca e litera mare
        print(caracter)           # afisam caracterul
```

Exemplul 2 – Numără vocalele dintr-un șir

Cerință: Scrie un program care citește un șir de la tastatură și afișează câte vocale conține.

Exemplu de rulare:

```
Input: "Informatica"
Vocale gasite: I, o, a, i, a
Output: Numar de vocale: 5
```

Caz particular:

```
Input: ""
Output: Numar de vocale: 0
Sirul este gol, se afiseaza valoarea 0.
```

Rezolvare pas cu pas:

1. Citim șirul de la tastatură.
2. Transformăm șirul în minuscule pentru a trata uniform literele mari și mici.
3. Inițializăm un contor la 0 pentru numărarea vocalelor.
4. Parcurgem șirul caracter cu caracter.
5. Verificăm dacă caracterul curent este vocală (a, e, i, o, u). Dacă caracterul curent este o vocală, incrementăm contorul.
6. Afișăm valoarea finală a contorului.

```

s = input("Introduceti un sir: ") # pas 1: citim sirul
s = s.lower()                     # pas 2: transformam in minuscule
contor = 0                        # pas 3: initializam contorul

for caracter in s:                # pas 4: parcurgem sirul
    if caracter in "aeiou":       # pas 5: verificam daca e vocala
        contor = contor + 1      # incrementam contorul

print("Numar de vocale:", contor) # pas 6: afisam rezultatul

```

Exemplul 3 – Înlocuiește spațiile cu liniuțe

Cerință: Citește un șir și afișează același șir cu spațiile înlocuite prin '-'.

Exemplu de rulare:

```

Input: "Salut ce mai faci"
Output: Salut-ce-mai-faci

```

Caz particular:

```

Input: "Salut lume"      (doua spatii intre cuvinte)
Output: Salut--lume
Daca sirul contine spatii consecutive, fiecare
spatiu este inlocuit individual.

```

Rezolvare pas cu pas:

1. Citim șirul de la tastatură.
2. Înlocuim ' ' cu '-'.
3. Afișăm rezultatul.

```

s = input("Introduceti un sir: ") # pas 1: citim sirul
rezultat = s.replace(" ", "-")    # pas 2: inlocuim spatiile
print(rezultat)                   # pas 3: afisam rezultatul

```

Probleme

1. Ce se afișează? (tracing de cod)

Se consideră următorul program Python:

```
s = "informatica"
rezultat = ""

for i in range(len(s)):
    if i % 2 == 0:
        rezultat = rezultat + s[i].upper()
    else:
        rezultat = rezultat + s[i]

print(rezultat)
```

Cerință: Scrie ce se afișează în urma executării programului.

Indicații:

- Parcurge șirul caracter cu caracter.
- Caracterul aflat la index par (0, 2, 4, ...) devine literă mare.
- Construiește șirul rezultat pas cu pas.

Răspuns:

2. Ce se afișează? (slicing + concatenare)

Se consideră următorul program Python:

```
s = "informatica"
rezultat = s[0:4] + s[4:8].upper() + s[8:]

print(rezultat)
```

Cerință: Scrie ce se afișează în urma executării programului.

Indicații:

- Împarte șirul în trei bucăți folosind slicing.
- Atenție la partea din mijloc, care este transformată în majuscule!

Răspuns:

3. Un șir se numește palindrom dacă se citește la fel de la stânga la dreapta și de la dreapta la stânga. Scrie un program Python care citește un șir și verifică dacă este palindrom.

Exemplu:

```
# Input: "radar"
# Output: Palindrom

# Input: "python"
# Output: Nu este palindrom
```

Indicații:

- Poți inversa șirul folosind `s[::-1]`.
- Compară șirul inițial cu cel inversat.
- Dacă sunt egale, atunci șirul este palindrom.

Rezolvare:

4. Citește un șir de caractere și afișează un nou șir în care toate vocalele au fost eliminate.

Exemplu:

```
# Input: "Informatica"
# Output: nfrmtc
```

Indicații:

- Parcurge șirul de caractere.
- Verifică dacă un caracter NU este vocală (a, e, i, o, u).
- Construiește un șir nou prin concatenare.

Rezolvare:

5. Citește o frază formată din mai multe cuvinte separate prin spațiu și afișează fraza în care fiecare cuvânt este inversat, păstrând ordinea cuvintelor.

Exemplu:

```
# Input: "Ana are mere"  
# Output: "anA era erem"
```

Indicații:

- Împarte fraza în cuvinte.
- Parcurge lista de cuvinte.
- Inversează fiecare cuvânt.
- Construiește o nouă frază, adăugând spații între cuvintele inversate.

Rezolvare:

6. Un agent secret primește un mesaj codificat format doar din **litere mici** și **cifre** (fără spații sau semne de punctuație). Scrie un program care:
- Afișează doar **literele** din mesaj.
 - Afișează doar **cifrele** din mesaj.
 - Construiește și afișează un **șir nou** în care:

- literele sunt transformate în **majuscule**;
- cifrele **pare** apar **de două ori** (ex: '4' → '44');
- cifrele **impare** rămân **nemodificate**.

Exemplu de rulare:

```
# Mesaj introdus: "a3b4c7d2"
# Cerinta 1 - Litere:  abcd
# Cerinta 2 - Cifre:   3472
# Cerinta 3 - Sir nou: A3B44C7D22
```

Indicații:

- Parcurge fiecare caracter din mesaj.
- Metodele `isalpha()` și `isdigit()` verifică tipul caracterului.
- Pentru cifre pare: convertește cu `int()` și verifică paritatea.
- Construiește șirul rezultat prin concatenare.

Rezolvare:

7. O furnică se află la poziția 0 pe o linie numerică. Primește un mesaj de navigare format doar din literele 'D', 'S' și cifre ('1'-'9'), după regulile:
- 'D' → furnica se mută **+1**, direcția curentă devine **dreapta**.
 - 'S' → furnica se mută **-1**, direcția curentă devine **stânga**.
 - **Cifră** → furnica se mută cu acel număr de pași **în direcția curentă**.
 - Direcția inițială este **dreapta**.

Afișează **poziția finală** a furnicii după parcurgerea întregului mesaj.

Exemplu de rulare:

```
# Mesaj: "D3SD2S"
# D  -> pozitie: +1  = 1    (directie: dreapta)
# 3   -> pozitie: +3  = 4    (directie: dreapta)
# S   -> pozitie: -1  = 3    (directie: stanga)
# D   -> pozitie: +1  = 4    (directie: dreapta)
# 2   -> pozitie: +2  = 6    (directie: dreapta)
# S   -> pozitie: -1  = 5    (directie: stanga)
# Pozitie finala: 5
```

Indicații:

- Folosește `pozitie = 0` și `directie = 1` (1 = dreapta, -1 = stânga).
- La 'D' sau 'S': actualizează `directie` și adaugă `directie` la `pozitie`.
- La cifră: adaugă `int(caracter) * directie` la `pozitie`.
- Folosește `isdigit()` pentru a distinge cifrele de litere.

Rezolvare: